Give me a summary of the functions present in https://github.com/Digital-Grinnell/common-DG-utilities

Here is a concise summary of the functions currently defined in `common_dg_utilities/dg_utils.py` and what they do.

ID and string utilities

- `generate_unique_id(page)`: Creates a session-scoped unique ID string like `dg_1729123456` based on the current epoch time, incrementing the timestamp if it collides with an ID already stored in `page.session.generated_ids`.
- `calculate_string_similarity(str1, str2)`: Returns a 0–100 integer "similarity" score using simple rules: exact match gives 100, substring matches give a length-ratio score, otherwise it counts common characters between the two strings and normalizes by the longer length.
- `sanitize_filename(filename)`: Normalizes a filename by replacing spaces with underscores, removing non word/hyphen/underscore/dot characters, collapsing odd underscore-around-hyphen patterns into a single hyphen, and reducing runs of underscores to a single underscore.

Fuzzy file search utilities

- `perform_fuzzy_search(base_path, target_filename, threshold=90)`: Walks `base_path` recursively to find the best matching file for `target_filename` using `calculate_string_similarity`; if the target has no extension, an exact basename match is boosted to at least 90 (slightly more for shorter extensions), and it returns `(best_match_path, best_match_ratio)` or `(None, 0)` on error.
- `perform_fuzzy_search_for_transcript(base_path, target_filename, threshold=90)`: Similar to `perform_fuzzy_search` but specialized for transcript workflows: it prioritizes media files (extensions like `.mp3`, `.mp4`, `.wav`, etc.) when matching and also looks for a `.csv` transcript file with the same basename, returning `(media_path, media_ratio, transcript_csv_path)`.
- `perform_fuzzy_search_batch(base_path, target_filenames, threshold=90, progress_callback=None, cancel_check=None, transcript_info=None)`: Runs fuzzy search over a list of filenames, optionally reporting fractional progress via `progress_callback` and supporting cancellation via `cancel_check`; entries marked as `transcript` (via `transcript_info[filename] == 'transcript'`) use the transcript-aware search, and the

function returns a results dict mapping each filename either to `(match_path, ratio)` or `(media_path, ratio, transcript_csv_path)`; it returns `None` if cancelled.

Content and config utilities

- `read_markdown(filename)`: Reads a markdown file and returns its contents as a string, logging and returning a markdown-formatted error message if the file is missing or another exception occurs.
- `read_config(page=None)`: Loads `_data/config.json` into a `config` dict, prints keys and values for debugging, and (intended to) store values into `page.session` if a `page` object is passed, then returns the config.

Session, messaging, and CSV validation helpers

- `session_get(page, key, default=None)`: Fetches a value from `page.session` and returns the provided default if the key is missing or the stored value is `None`.
- `show_message(page, text, is_error=False)`: Logs a message via a `logger` stored in `page.session["logger"]` and shows it in a Flet `SnackBar` on the page, using red background for errors and green for non-errors, then calls `page.update()`.
- `validate_csv_headings(csv_file_path, mode)`: For CollectionBuilder mode, loads the verified headings from `_data/verified_CSV_headings_for_GCCB_projects.csv`, reads just the header row of the given CSV (trying several encodings), computes which CSV column names are not in the verified list, and returns `(True, unmatched_headings, None)` on success (permissive: extras allowed) or `(False, [], error_message)` if the verified file or CSV is missing or another error occurs.

Would you like a short usage example for any one of these (for example, how you'd wire `perform_fuzzy_search_batch` into a Flet workflow)?